

Projeto 3 - Virtual Memory

Prof. Eduardo Tavares
eagt@cin.ufpe.br

Projeto 3

Frame Table (Tabela de Molduras)

Supplemental Page Table (Tabela de Páginas Suplementar)

Swapping

Memory Mapped Files (Arquivos Mapeados na Memória)

Projeto 3

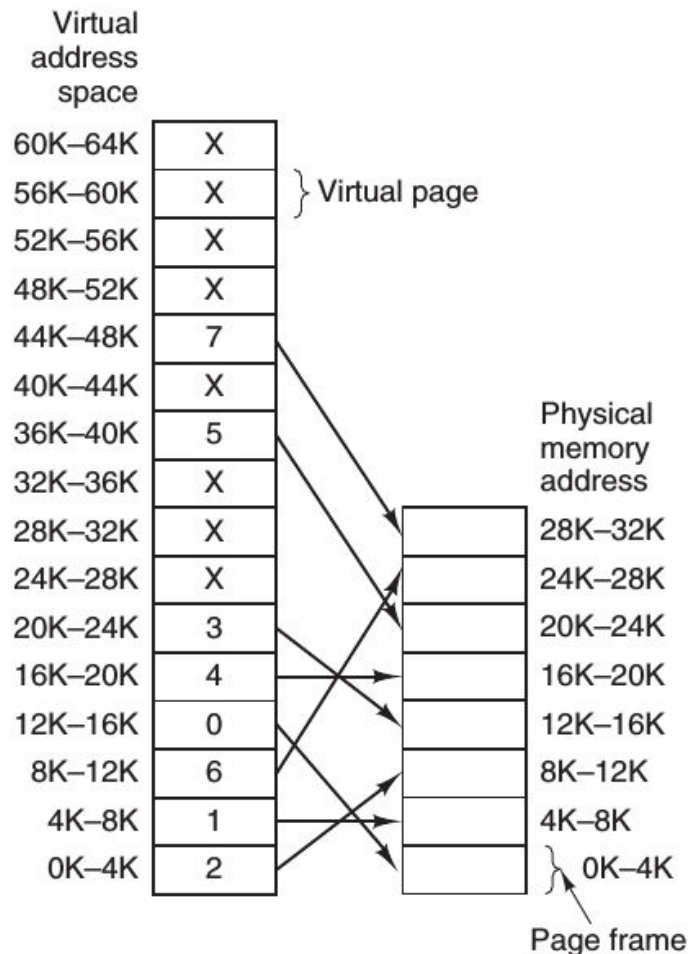
Trabalhar em `/vm` e `/userprog`

Há dependência forte com o projeto 2

Bugs no projeto 2 podem afetar o projeto atual

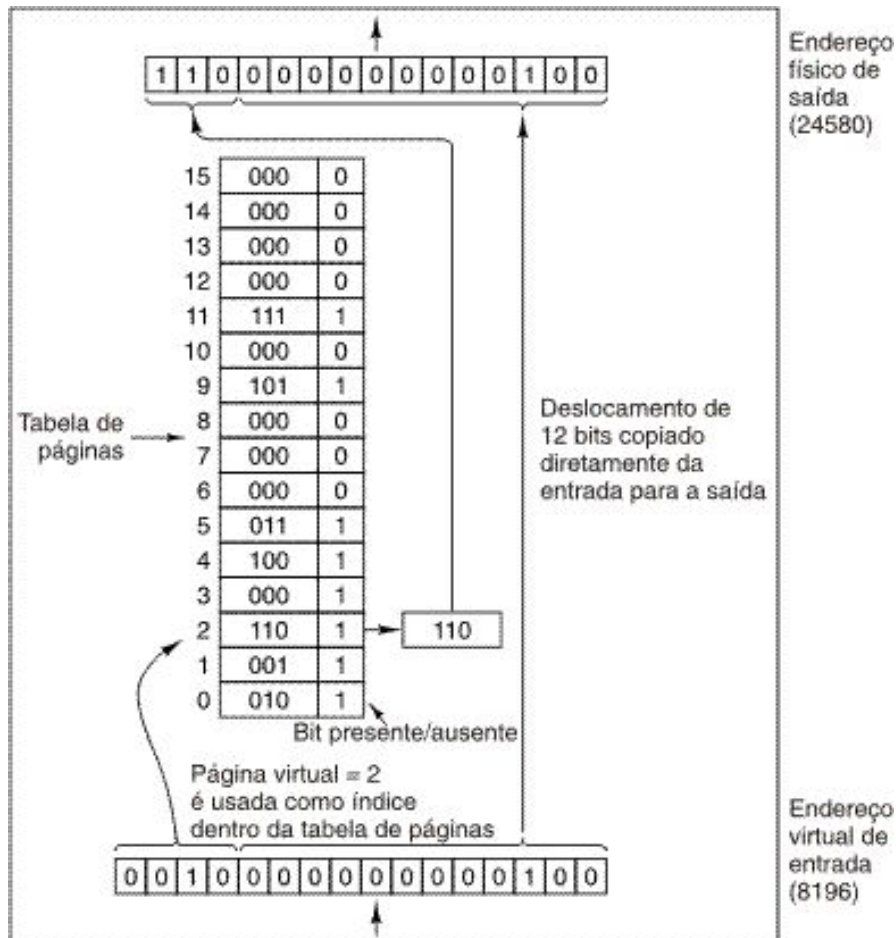
Faça o *copy* e *paste* do projeto 2 para fazer o projeto 3

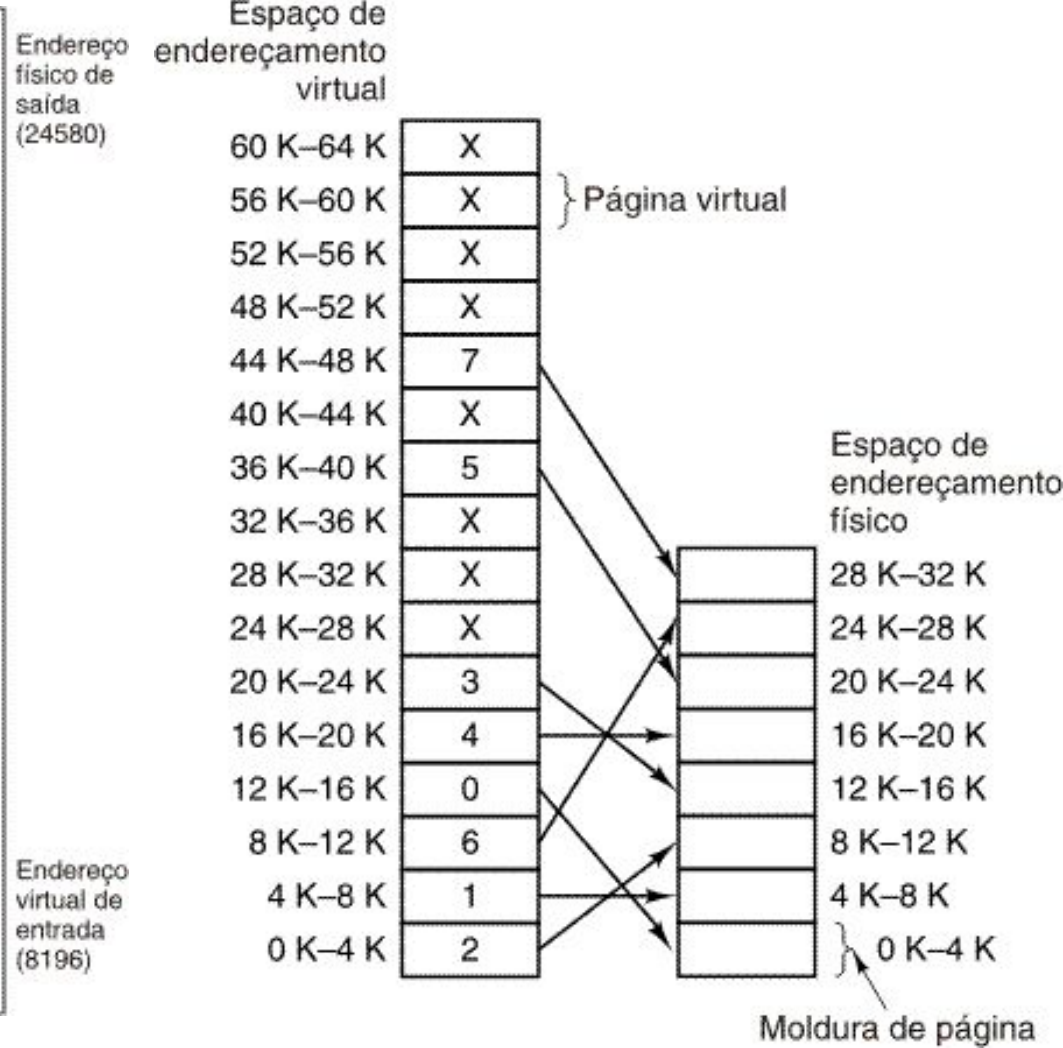
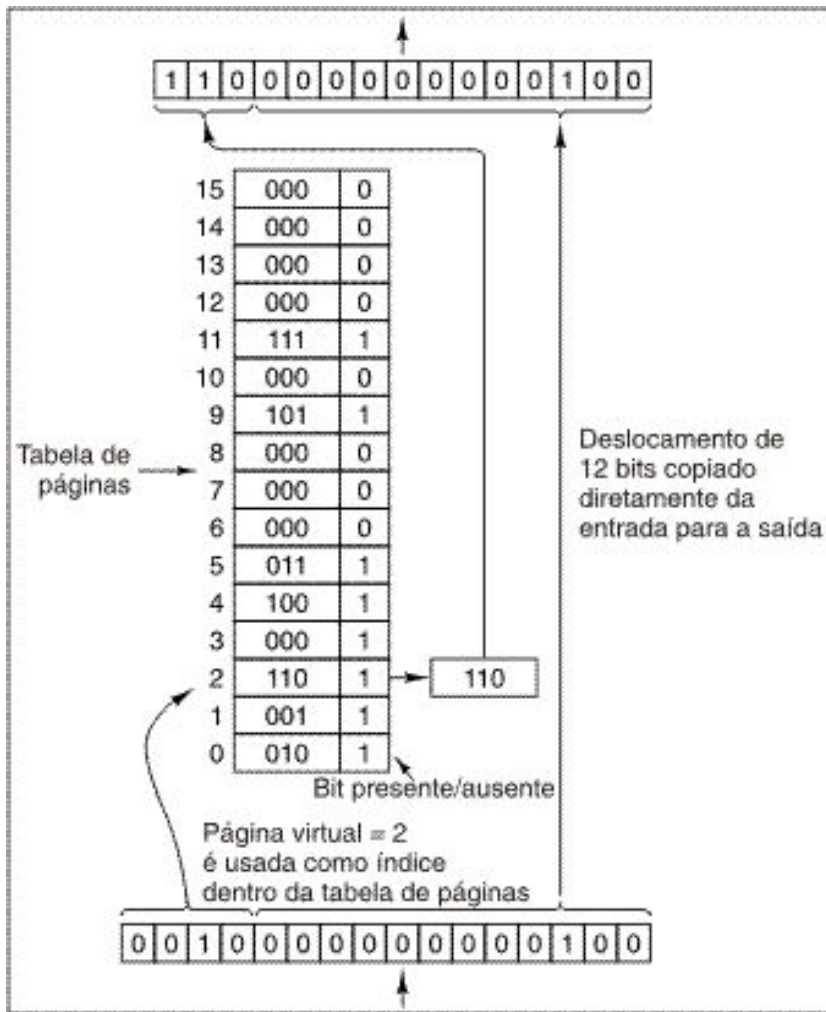
Memória Virtual



Memória Virtual

Uma tabela de
página por
processo



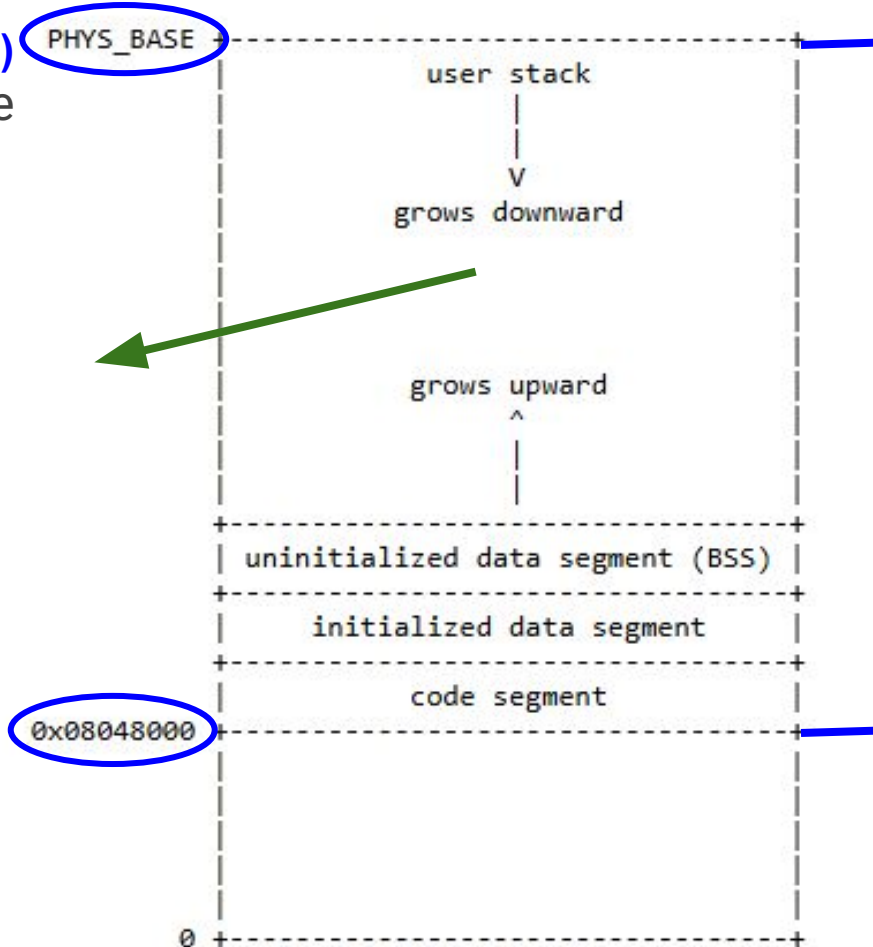


< 0xc0000000 (3 GB)

Início do espaço de
endereçamento do
Kernel (até 4 GB)

O tamanho da
pilha pode
crescer no
projeto 3.

Aproximadamente
128MB. Segue um
padrão do System
V. Precisa ser maior
que o endereço.



Espaço de
endereçamento
válido do
processo do
usuário

Memória Física

Para as páginas do usuário, depende do mapeamento com as molduras(*frames*)

O mapeamento das páginas do *kernel* são de 1-para-1 na memória

1GB

0x0

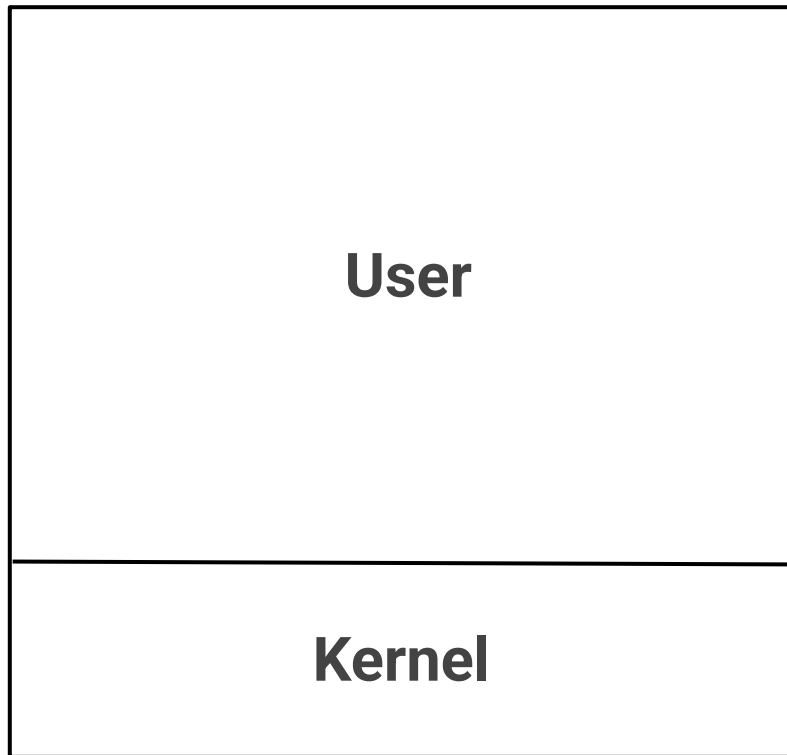
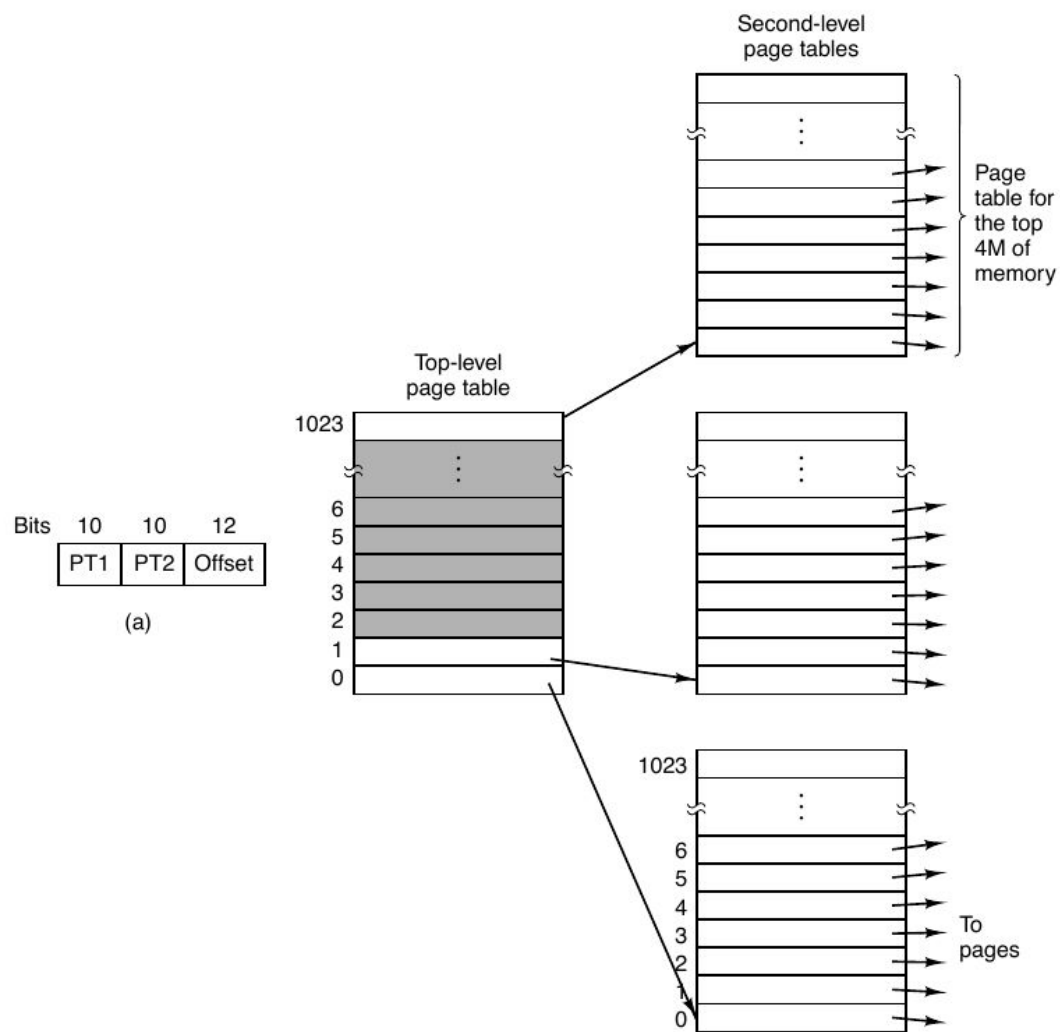


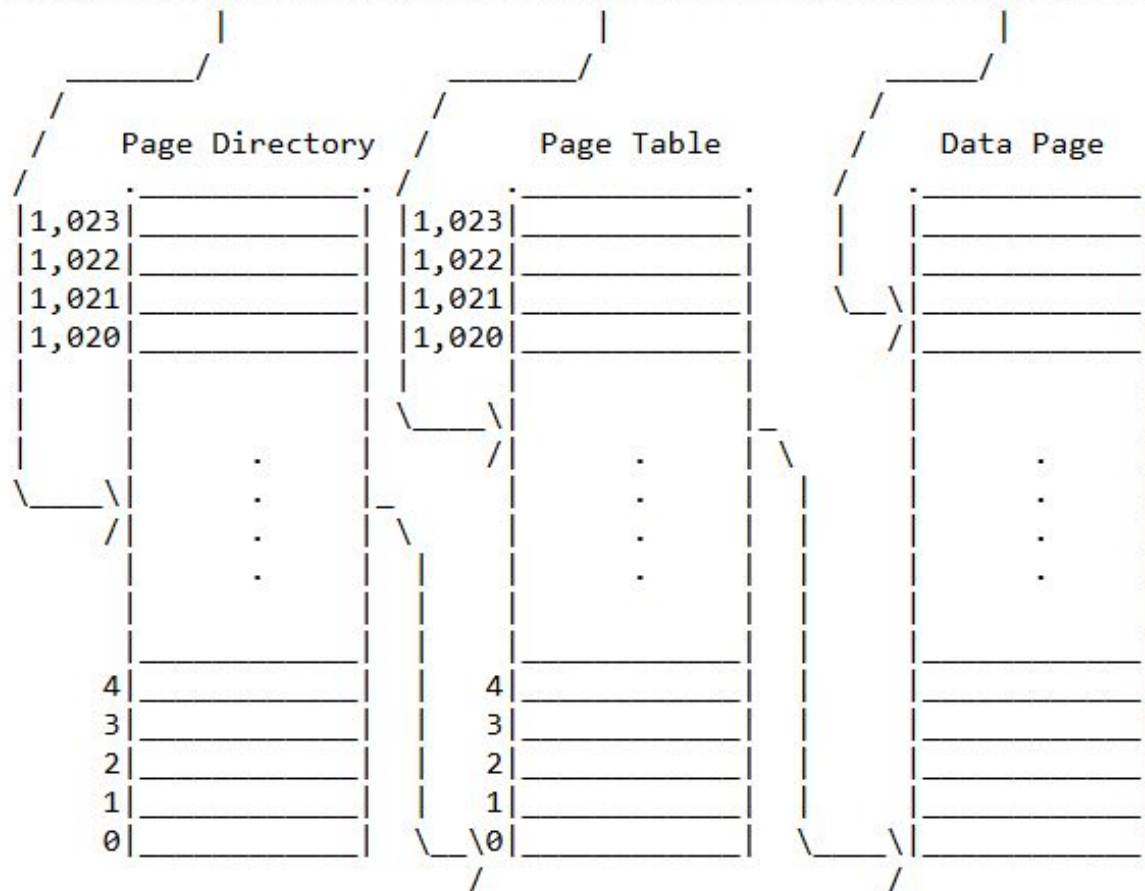
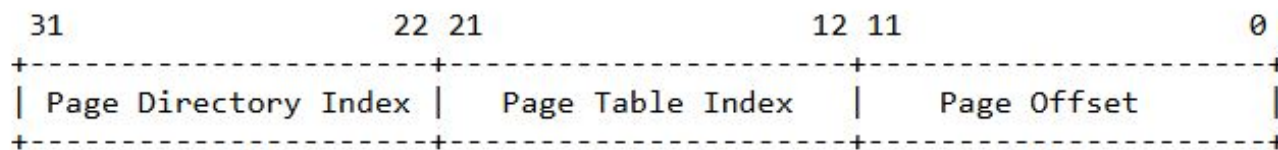
Tabela de Páginas

A arquitetura x86 usa uma tabela de páginas multinível (2 níveis)

Nível 1 - *Page Directory* - Uma entrada (*Page Directory Entry*) aponta para uma tabela de páginas

Nível 2 - *Page Table* - Guarda um subconjunto de páginas do processo





Page Pools

80x86 não possui mecanismos para trabalhar diretamente com a memória

Pintos trabalha com 2 grupos de páginas: *kernel pages* e *user pages*

Representam os *frames* da memória

Page Pools

`malloc.h`

`malloc_get_page (FLAG)` – Pega uma página

`malloc_get_page (PAL_USER | PAL_ZERO) ;` – Pega uma
página de usuário limpa

`malloc_get_page (0)` – Pega uma página de *kernel*

Page Pools

Pegou uma página, precisa colocar na tabela de página do processo

`install_page` em `processo.c` que usa `pagedir_set_page` em `pagedir.h`

`pagedir_clear_page` tira a página da tabela de páginas

`palloc_free_page` devolve a página para o respectivo *pool*

Atenção à ordem!

Projeto 3

Se memória virtual está implementada, qual é o objetivo do projeto?

Projeto 3

Se memória virtual está implementada, qual é o objetivo do projeto?

1. Não tem tratamento de *page fault*
2. Não tem substituição de páginas (ex: memória cheia)
3. Não tem swapping
4. Não permite crescimento da pilha
5. Não tem arquivo mapeado na memória

Projeto 3

Tabela de molduras (*frame table*) será usada para acompanhar o uso dos *frames* (***user pages***). Usada pelo algoritmo de substituição

Tabela de páginas suplementar (*supplemental page table*) indica páginas virtuais do processo que estão na memória ou não (*swapping*). Outros atributos para o algoritmo de substituição

Page Fault

Precisa implementar o tratamento em `userprog/exceptions.c`:

```
static void page_fault (struct intr_frame *f)
```

Algumas razões da falta de página:

1. Crescimento da pilha (precisa criar a página e alocar *frame*)
2. Página ausente na memória (está na área de *swap*)
3. Acesso indevido de posição memória;

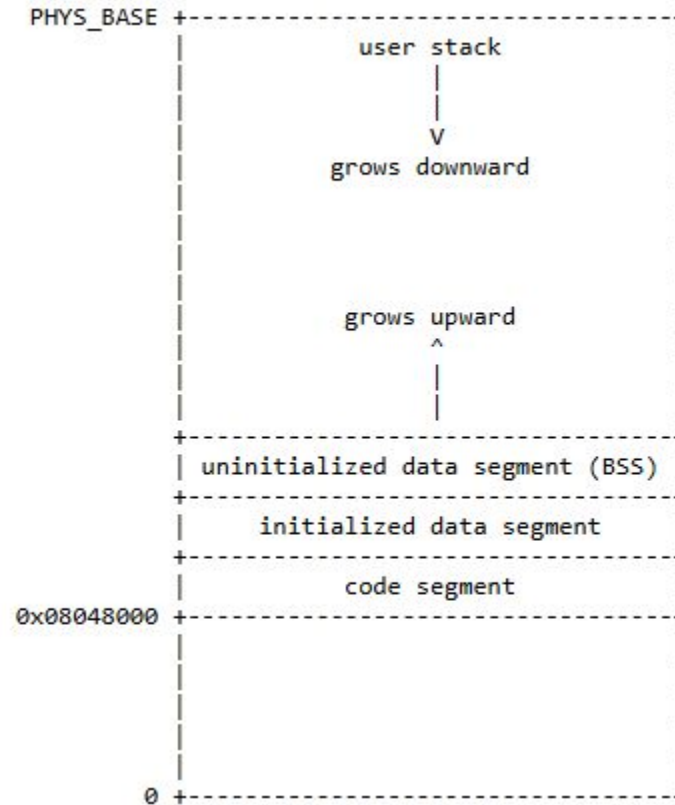
Crescimento da Pilha (Stack Growth)

Chamada de funções fazem a pilha crescer

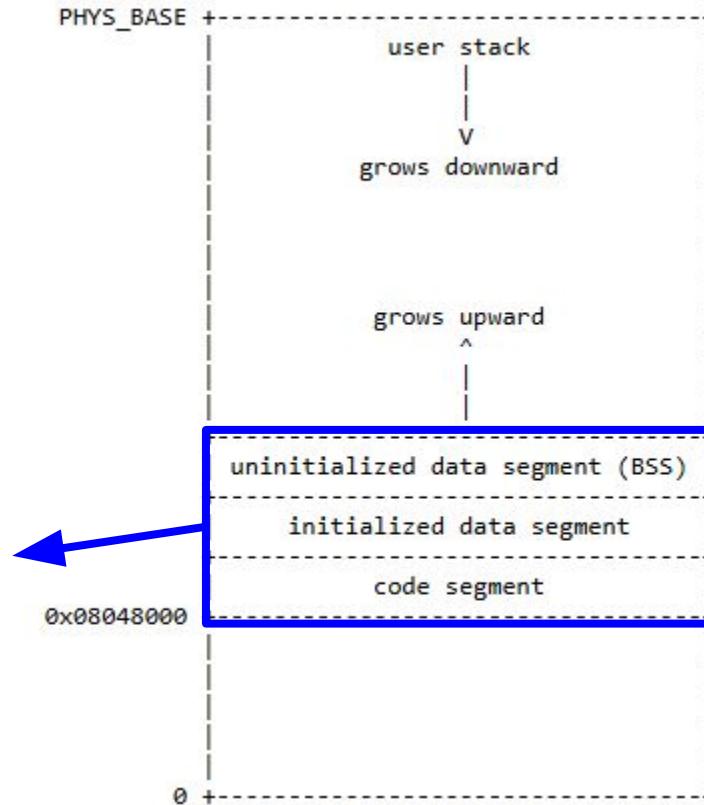
Detectar que uma falta de página é o crescimento da pilha

Alocar memória (criando *página* e usando *frame*)

Crescimento da Pilha (Stack Growth)



Crescimento da Pilha (Stack Growth)

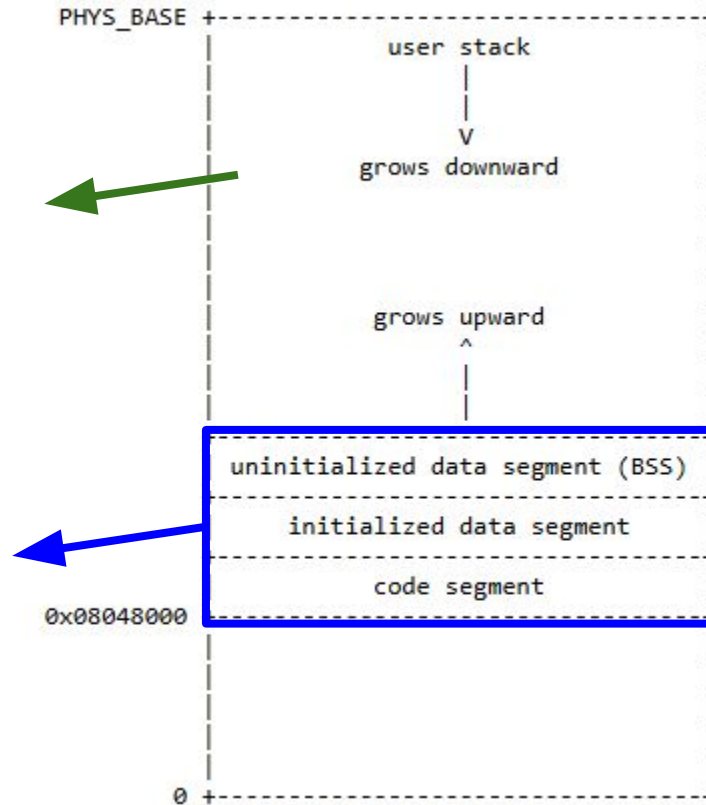


Páginas criadas na
inicialização do
processo. Falta de
página é devido ao
swapping

Crescimento da Pilha (Stack Growth)

Assumir que endereço `esp - 32` bytes (máximo) é endereço válido. Falta de página sem página criada, é mais memória necessária

Páginas criadas na inicialização do processo. Falta de página é devido ao *swapping*



Swapping

Uso de `devices/block.h`

Nos testes, área de swap é criada na inicialização do S.O.

O tamanho da partição pode ser obtido com as funções do *driver*

Precisa gerenciar os *slots* ocupados e livres (`bitmap`)

Memory Mapped File

System calls: `mapid_t mmap(int fd, void* addr);` e `void munmap(mapid_t mapping);`

O arquivo é a “área de *swap*” das páginas

Usar `reopen` e guardar a referência do arquivo.

Salvar no arquivo somente se a página estiver *dirty*

(`pagedir_is_dirty`)

Carregar as páginas somente quando forem acessadas

Sugestão

1. Leia a documentação oficial
2. Leia o guia CSCI 350 (excelente!)
3. Comece com a tabela de *frames* acompanhando a alocação dos ***user pages***
4. Em seguida, faça a tabela de páginas e o mapeamento
5. Implemente *swapping*

Sugestão

6. Para o algoritmo de substituição, pode considerar a página mais antiga com `timer_ticks()`
7. Faça o mapeamento de arquivos na memória por último
8. Faça entregas intermediárias dos projetos

Arquivos

Criar arquivos em /vm: `frame.c`, `frame.h`, `page.c`, `page.h`

`swap.c` e `swap.h`

Alterar `/src/Makefile.build` para colocar os novos arquivos na
compilação